

Escape the Cursed Mansion

Game Overview — A horror survival adventure built with clean architecture and powerful design patterns

TEAM: RANEEM ALHAMEED / SIDRA WALI / RAHAF ATMEH

PROJECT: CURSED MANSION



Project Overview

Players navigate a haunted mansion, battling enemies, collecting loot, and solving puzzles — all to escape before the curse consumes them.

Objective

Escape the mansion by defeating enemies and solving puzzles across progressively dangerous rooms.

Core Mechanics

Inventory management, weapon combat, enemy encounters, and loot collection.

Architecture

Built on the Composite Pattern for a flexible, scalable item and inventory system.



Gameplay Features



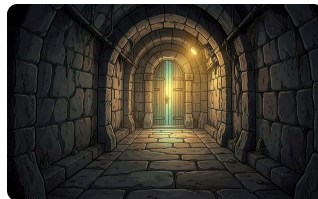
Combat System

Players equip weapons and engage enemies in turn-based or real-time combat with damage calculations and durability.



Loot & Inventory

Collect items, stack them in containers, and manage a hierarchical inventory with nested item groups.



Level Progression

Advance through mansion rooms, each introducing new enemy types, puzzles, and item challenges.

Code Explanation

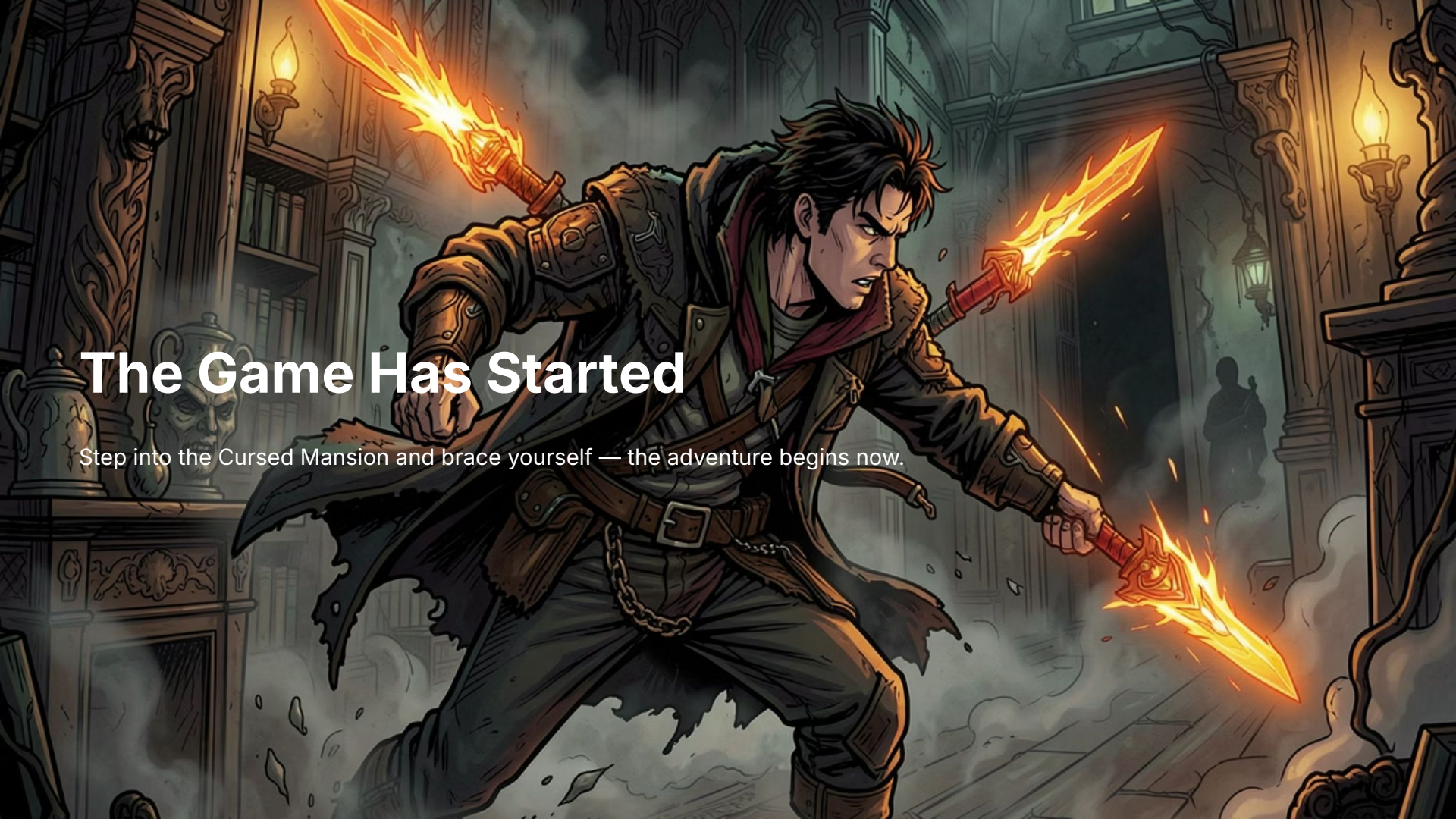
```
// Abstract Component
abstract class GameItem {
    void addItem(GameItem i) {}
    void removeItem(GameItem i) {}
    void display() {}
    int getWeight() { return 0; }
}

// Leaf
class SimpleItem : GameItem {
    string name; int weight;
    void display() {
        print(name + " (" + weight + "kg)");
    }
}

// Composite
class ItemContainer : GameItem {
    List<GameItem> children;
    void addItem(GameItem i) {
        children.add(i);
    }
    void display() {
        for (GameItem c : children)
            c.display();
    }
}
```

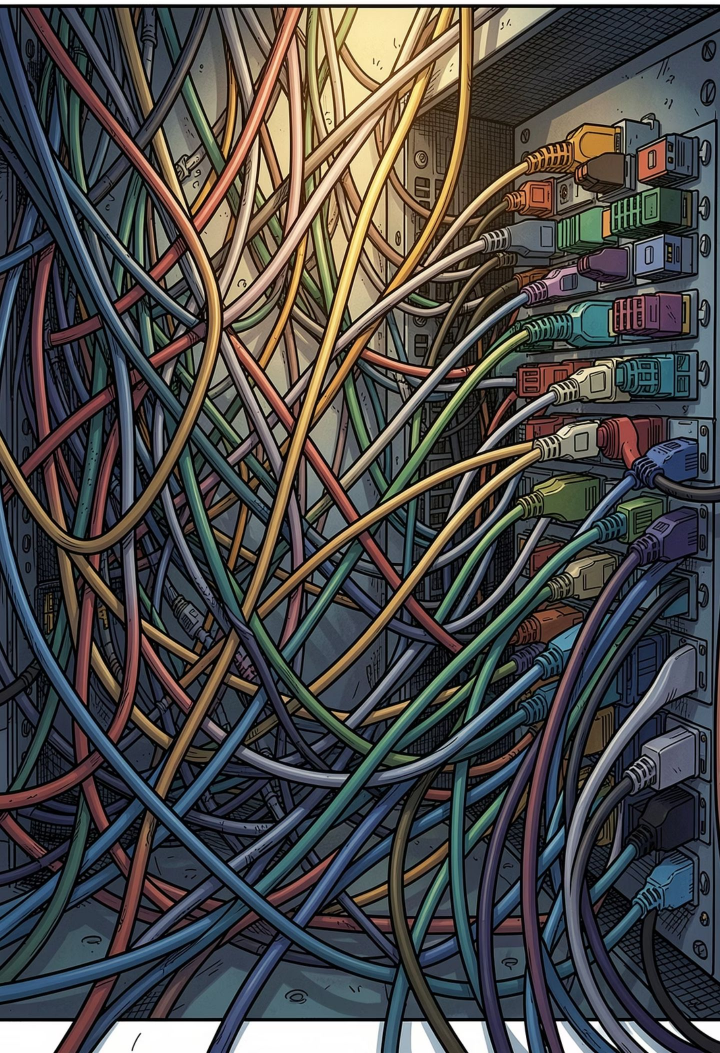
Key Responsibilities

- **GameItem** — Defines the contract for all items in the system.
- **SimpleItem** — Leaf node; stores name, weight, and combat stats.
- **ItemContainer** — Composite node; manages child items and delegates operations recursively.



The Game Has Started

Step into the Cursed Mansion and brace yourself — the adventure begins now.



Problem & Solution

✗ The Problem

Managing complex item structures — bags inside chests, stacks of potions, nested equipment — required repetitive conditional logic. Every new container type meant rewriting traversal and display code.

✓ The Solution

The Composite Pattern unified SimpleItems and ItemContainers under one interface. Adding, removing, and displaying items works identically whether the item is a single sword or a bag of 20 objects.



DESIGN PATTERN

The Composite Pattern

The Composite Pattern lets clients treat **individual objects** and **compositions of objects** uniformly. In our game, it powers the entire inventory system.

Why We Chose It

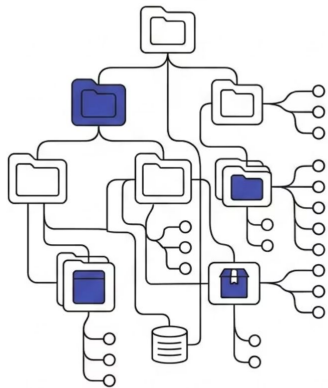
Items can be simple (a sword) or complex (a bag containing multiple items). The pattern handles both seamlessly.

Key Benefit

No special-case code for nested containers — the system treats every item the same way.

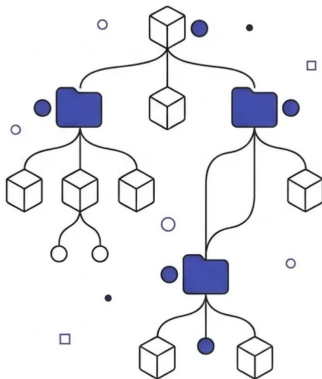
Trade-offs: Traditional vs. Composite

TRADITIONAL APPROACH



Requires separate classes
Complex nested if-statements
Hard to extend
Tightly coupled code

COMPOSITE PATTERN



Single interface
Clean recursive traversal
Easy to add item types
More memory overhead

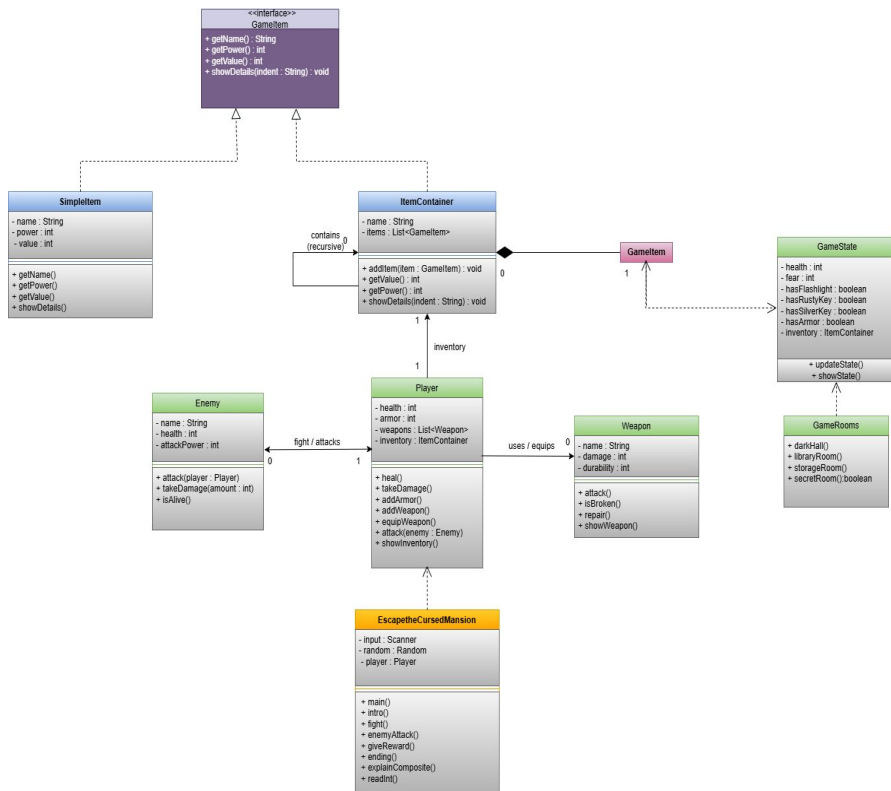
Our Verdict

The Composite Pattern introduces minimal overhead while dramatically improving maintainability. For a game where inventory complexity grows over time, the pattern pays for itself quickly.



Trade-off: Slightly higher memory usage from object allocation
— acceptable given the clarity and scalability gains.

UML Diagram — Composite Structure



How It Works

GamelItem is the abstract base defining the contract. **SimpleItem** represents leaf nodes — weapons, potions, keys. **ItemContainer** is the composite that holds child items and delegates operations recursively.

i This structure allows infinitely nested containers without changing client code — a backpack inside a chest inside a room.

Note

Composite Pattern
GamelItem =
Component
SimpleItem = Leaf
ItemContainer =
Composite

Thank you

Thank you to the team and to everyone who joined us for this journey.
We appreciate your support, your time, and your excitement as we built this together.

